

GRADUATION THESIS

Implementation and Extension of a Visual-Inertial Odometry Pipeline with Hover-Aware FIFO/LIFO Supervisory Estimation

[Candidate Name] | [Department] | [University] | [Year]

RECOMMENDED THESIS STRUCTURE

The following chapter outline is recommended for this graduation thesis. Each chapter is listed with its purpose and key content.

Table 0.1: Recommended Thesis Chapter Structure

Chapter	Title	Purpose & Key Content
Title Page	—	Thesis title, candidate name, institution, supervisor, date, degree.
Abstract	Abstract (200–300 words)	Problem, approach, key technical contributions, results. Written last.
Chapter 1	Introduction	Motivation for VIO, problem statement, UAV hover challenge, scope, thesis organisation.
Chapter 2	Background and Related Work	EKF fundamentals, MSCKF theory, OpenVINS literature, VIO state-of-the-art, hover/low-excitation work.
Chapter 3	System Architecture	High-level VIO pipeline design, module overview, ROS integration, threading model, coordinate frames.
Chapter 4	Methodology (Core)	State estimation math, propagation, MSCKF update, SLAM, ZUPT, and the full FIFO/LIFO hover framework. [Detailed in this document]
Chapter 5	Implementation	Python-specific design decisions, module-by-module code walk, key deviations from C++ OpenVINS.
Chapter 6	Experiments and Evaluation	Datasets, metrics (ATE, RTE, NEES), comparison against baseline, ablation of LIFO/ZUPT.
Chapter 7	Discussion	Analysis of results, failure modes, hover mode behaviour, FIFO vs LIFO tradeoffs, limitations.
Chapter 8	Conclusion and Future Work	Summary of contributions, open problems, future directions.
References	Bibliography	IEEE/APA format, all cited

		works.
Appendix A	Mathematical Derivations	Full Jacobian derivations, quaternion algebra proofs.
Appendix B	Configuration Reference	YAML key mapping, tuning guide.
Appendix C	Code Structure	Repository topology, module dependency diagram.

CHAPTER 3: SYSTEM ARCHITECTURE

3.1 Overview

The implemented system is a complete Visual-Inertial Odometry (VIO) pipeline written in Python, designed to run within the Robot Operating System (ROS) framework. The pipeline fuses measurements from an Inertial Measurement Unit (IMU) and one or two cameras to produce real-time pose estimates suitable for Unmanned Aerial Vehicle (UAV) applications, including scenarios involving prolonged near-stationary hover.

The architecture follows the error-state Extended Kalman Filter (EKF) paradigm established in the Multi-State Constraint Kalman Filter (MSCKF) literature, extended with an original supervisory mode controller that adapts estimation behaviour to hover conditions. Figure 3.1 illustrates the high-level module decomposition.

Figure 3.1: High-level architecture of the VIO pipeline. Dashed boxes denote optional modules; the FIFO/LIFO controller is the primary original contribution of this work.

3.2 Module Decomposition

The pipeline consists of six major subsystems:

- **ROS Interface Layer:** Receives raw IMU and camera messages, manages rate limiting, maintains a timestamp-sorted camera queue, and dispatches camera frames to the update worker.
- **Front-End Tracker (TrackKLT):** Performs KLT optical flow tracking, RANSAC outlier filtering, and maintains a persistent FeatureDatabase keyed by feature identity.
- **Inertial Initializer:** Monitors the IMU and feature buffers, applies static variance gating, and recovers gravity-aligned orientation and initial biases before the filter is activated.
- **Propagator:** Integrates IMU measurements forward in time using one of three selectable methods, updates the state and covariance, and manages the sliding clone window.

- Updater Suite: Three independent EKF measurement updaters — MSCKF for marginalised feature constraints, SLAM for persistent landmark tracking, and ZUPT for zero-velocity enforcement.
- VioManager Orchestrator: Coordinates all subsystems, enforces the FIFO/LIFO mode machine, and publishes pose estimates to ROS topics.

3.3 Coordinate Frames and Conventions

Four reference frames are used throughout the pipeline:

- G (Global/Inertial): A fixed world frame. Gravity is aligned with the negative z-axis: $g = [0, 0, g_mag]^T$.
- I (IMU Body): The IMU sensor frame, co-moving with the vehicle.
- C (Camera): The optical frame of each camera; multiple cameras share an extrinsic calibration to I.
- A (Anchor): A selected historical clone pose used as the reference for anchored landmark representations.

All quaternions use the JPL convention with layout $[x, y, z, w]$. A quaternion q_{GtoI} encodes a rotation from the global frame to the IMU frame. The standard EKF error-state perturbation for orientation uses the left-multiplicative small-angle approximation.

3.4 Threading and Data Flow

The node operates two concurrent threads. The IMU callback thread runs at full sensor rate (typically 200–400 Hz), buffers inertial data, propagates a fast odometry estimate for immediate publication, and triggers the camera worker when idle. The camera worker thread drains a timestamp-sorted queue, processing each frame only when sufficient IMU data exists to guarantee temporal alignment. This decoupled design gives low-latency IMU-rate odometry while serialising the more expensive camera update pipeline.

CHAPTER 4: METHODOLOGY

This chapter presents the complete mathematical and algorithmic methodology underlying the implemented VIO pipeline. It is organised into seven sections: the state representation and error-state formulation (Section 4.1), the IMU propagation model (Section 4.2), the visual front-end (Section 4.3), the MSCKF measurement update (Section 4.4), SLAM landmark management (Section 4.5), zero-velocity updating (Section 4.6), and the hover-aware FIFO/LIFO supervisory estimation framework — the primary original contribution of this work (Section 4.7).

4.1 State Representation and Error-State Formulation

4.1.1 Nominal State Vector

The filter maintains a nominal (mean) state x composed of a mandatory IMU block and a set of dynamically sized blocks for camera clones and optional SLAM landmarks:

$$x = [x_{IMU} \mid x_{clone_1} \cdots x_{clone_N} \mid x_{L_1} \cdots x_{L_M}]^T$$

The IMU block contains fifteen scalar degrees of freedom:

$$x_{IMU} = [q_{GtoI}, p_{IinG}, v_{IinG}, b_g, b_a]$$

where $q_{GtoI} \in S^3$ is the JPL orientation quaternion (4 parameters, 3 error-state DOF), $p_{IinG} \in \mathbb{R}^3$ is the IMU position in the global frame, $v_{IinG} \in \mathbb{R}^3$ is the velocity, $b_g \in \mathbb{R}^3$ is the gyroscope bias, and $b_a \in \mathbb{R}^3$ is the accelerometer bias.

When online calibration is enabled, additional blocks are appended for camera time-offset $dt_{CAMtoIMU}$ (1 DOF), per-camera extrinsic pose T_{ItoC} (6 DOF each), per-camera intrinsics (8 DOF each), and IMU intrinsic matrices D_w, D_a, T_g .

4.1.2 Error-State and Covariance

The error-state δx is defined as the difference between the true state and the nominal estimate. For vector quantities the error is additive; for orientations the left-multiplicative perturbation is used:

$$\delta q = [\frac{1}{2} \delta \theta, 1]^T, \quad q_{\text{true}} = \delta q \otimes q_{\text{nominal}}$$

The filter propagates the covariance matrix $P \in \mathbb{R}^{(n \times n)}$ of the error-state. The total dimension n is:

$$n = n_{\text{IMU}} + n_{\text{calib}} + 6 \cdot N_{\text{clone}} + n_{\text{SLAM}}$$

where $n_{\text{IMU}} = 15$, n_{calib} sums all enabled calibration DOF, N_{clone} is the current number of active camera clones, and n_{SLAM} is the sum of landmark DOF in the state.

4.1.3 IMU Intrinsic Models

Two IMU intrinsic parameterisations are supported. The KALIBR model uses a lower-triangular 3×3 matrix D_a (or D_w), while the RPNG model uses an upper-triangular form. Both are stored as 6-vectors in the state and reconstructed during Jacobian computation. A gravity sensitivity matrix $T_g \in \mathbb{R}^{(3 \times 3)}$ models the coupling between acceleration and apparent angular rate, stored as a 9-vector assembled column-wise.

4.2 IMU Propagation

4.2.1 Corrected Measurements

Raw IMU measurements ω_m and a_m are corrected for scale, misalignment, and bias before integration. Given intrinsic matrices D_a and D_w , alignment rotations R_{ACCtoIMU} and $R_{\text{GYROtoIMU}}$, and current bias estimates:

$$\begin{aligned} \hat{a} &= R_{\text{ACCtoIMU}} \cdot D_a \cdot (a_m - b_a) \\ \hat{\omega} &= R_{\text{GYROtoIMU}} \cdot D_w \cdot (\omega_m - b_g - T_g \cdot \hat{a}) \end{aligned}$$

The corrected gyroscope signal $\hat{\omega}$ includes gravity sensitivity correction, which is especially significant for low-quality MEMS IMUs.

4.2.2 Continuous-Time Dynamics

The nominal state evolves according to the kinematic equations on the IMU manifold:

$$\begin{aligned}\dot{q} &= \frac{1}{2} \Omega(\hat{\omega}) q \\ \dot{p} &= v \\ \dot{v} &= R(q)^T \hat{a} - g \\ \dot{b}_g &= \eta_{wg}, \quad \dot{b}_a = \eta_{wa}\end{aligned}$$

where $\Omega(\omega)$ is the 4×4 skew-symmetric matrix for quaternion kinematics, $R(q)$ is the rotation matrix from q , $g = [0, 0, g_{\text{mag}}]^T$ is gravity in the global frame, and η_{wg} , η_{wa} are zero-mean white noise processes modelling bias random walks.

4.2.3 Integration Methods

Three numerical integration methods are configurable at runtime:

- Discrete (Euler): First-order approximation. Efficient, suitable for high-rate IMU. Accumulated error is acceptable when IMU rate exceeds 200 Hz.
- Runge-Kutta 4 (RK4): Classic four-stage integration. Significantly reduces integration error for larger time steps. Interpolated IMU terms are used at intermediate stages.
- Analytical: Closed-form integration using precomputed ξ_1 through ξ_4 exponential map coefficients. Provides near-exact integration of the piecewise constant angular velocity assumption.

4.2.4 Error-State Transition

For each IMU sub-interval $[t_k, t_{k+1}]$ with duration Δt , the linearised error-state transition matrix $F \in \mathbb{R}^{(15 \times 15)}$ and noise input matrix $G \in \mathbb{R}^{(15 \times 12)}$ are computed. The discrete process noise covariance is:

$$Q_d = G Q_c G^T$$

where Q_c is the continuous-time noise covariance constructed from the noise densities σ_ω , σ_a and random-walk intensities σ_{wb} , σ_{ab} . Across a full camera interval, transitions are composed by matrix product:

$$\Phi_{total} = F_n \cdot F_{n-1} \cdot \dots \cdot F_1$$

$$Q_{total} = F_n(\dots(F_2 Q_1 F_2^T + Q_2)\dots)F_n^T + Q_n$$

A single EKF propagation call then applies the summed transition to the full state covariance, including cross-covariance terms between the IMU block and clone/SLAM blocks.

4.2.5 FIFO Clone Augmentation vs. LIFO Propagation-Only

After IMU propagation, the pipeline diverges based on the current operating mode. In FIFO mode, a new pose clone is appended to the state via stochastic clone augmentation: the clone copies the current IMU orientation and position, and the covariance is expanded to include full cross-correlations with the camera time-offset calibration state. In LIFO mode, propagation-only is used — the IMU state and covariance are updated but no new clone is appended. This preserves the existing clone window and is the foundation of the LIFO hover strategy described in Section 4.7.

4.3 Visual Front-End

4.3.1 KLT Optical Flow Tracking

Visual feature tracking is performed using the Lucas-Kanade (KLT) optical flow algorithm with image pyramids. A five-level pyramid is constructed from each incoming frame, and correspondences are sought within a 15×15 pixel search window at each level. This multi-scale approach gives robustness to small image displacements during hover as well as larger displacements during dynamic flight.

When the tracked feature count falls below a configurable threshold, new feature points are detected using FAST corner detection on a spatial grid of configurable resolution, ensuring uniform image coverage. Optional histogram pre-processing (CLAHE or global equalisation) improves detection in high-contrast or low-contrast scenes.

4.3.2 Outlier Rejection

After KLT propagation, geometric consistency is enforced using RANSAC estimation of the fundamental matrix on undistorted normalised coordinates. Correspondences inconsistent with the estimated geometry are marked as outliers and removed from the active set. This two-stage process — KLT tracking followed by geometric verification — yields a robust set of inlier correspondences suitable for EKF updating.

4.3.3 Feature Database

Accepted correspondences are inserted into a persistent FeatureDatabase indexed by feature identity. Each feature record stores separate per-camera arrays of timestamps, distorted pixel coordinates, and undistorted normalised coordinates. The database supports efficient query operations used by the estimator: features not observed after a given timestamp (marginalised tracks), features with observations spanning a required time interval, and targeted single-feature retrieval. This design enables clean separation between the front-end tracking logic and the back-end estimation scheduling.

4.3.4 Triangulation and Refinement

Before features are used in EKF updates, their 3D positions must be initialised via triangulation. Linear triangulation assembles a normal equation system from the bearing orthogonality constraint across all observations:

$$A = \sum_i (B_i^\perp)^T B_i^\perp, \quad b = \sum_i (B_i^\perp)^T B_i^\perp p_{\{C_i\}}^A$$

$$p_{F^A} = A^{-1} b$$

where $B_i^\perp = I - \hat{b}_i \hat{b}_i^T$ is the orthogonal projector onto the plane perpendicular to bearing $\hat{b}_i$. A condition number check and depth range validation gate the linear solution quality. For features with a dominant depth direction, a 1D triangulation along the anchor bearing is used as a numerically more stable alternative.

The linear estimate is then refined with a Levenberg-Marquardt (LM) iteration over the inverse-depth parameterisation (α, β, ρ) . The LM damping factor λ is adapted based on the cost reduction at each step, and convergence is declared when the parameter increment norm falls below a configured tolerance.

4.4 MSCKF Measurement Update

4.4.1 Principle

The Multi-State Constraint Kalman Filter (MSCKF) exploits the geometric consistency of a feature track across multiple camera poses without requiring the feature to be maintained in the filter state. A track spanning K camera clones provides $2K$ pixel observations but adds no new state variables. The feature position is instead eliminated from the linearised update by a nullspace projection, yielding a constraint entirely in terms of the clone poses already in the state.

4.4.2 Measurement Jacobian

For a feature f observed at clone pose i , the predicted observation is:

$$z_i = \pi(R_{ItoC} \cdot (R_{\{GtoI_i\}} (p_{F^A}^G - p_{\{I_i\}}^G) - p_{CinI}))$$

where $\pi(\cdot)$ is the camera projection and distortion function. Linearising around the current nominal state yields Jacobians $H_{\{x,i\}} \in \mathbb{R}^{(2 \times n)}$ with respect to the full state, and $H_{\{f,i\}} \in \mathbb{R}^{(2 \times 3)}$ with respect to the feature position. Stacking over all K observations produces a system:

$$r \approx H_x \delta x + H_f \delta f + n$$

4.4.3 Nullspace Projection

The feature Jacobian $H_f \in \mathbb{R}^{(2K \times 3)}$ is factored using a thin QR decomposition: $H_f = [Q_1, Q_2] \begin{bmatrix} T \\ 0 \end{bmatrix}$, where the columns of Q_2 span the left nullspace of H_f^T . Pre-multiplying the linearised system by Q_2^T removes the feature term:

$$\tilde{r} = Q_2^T r, \quad \tilde{H}_x = Q_2^T H_x, \quad \tilde{n} = Q_2^T n$$

The projected system $\tilde{r} \approx \tilde{H}_x \delta x + \tilde{n}$ is a pure state constraint. Measurement noise $\tilde{n}$ is Gaussian with covariance $\tilde{R} = \sigma_{\text{pix}}^2 Q_2^T Q_2 = \sigma_{\text{pix}}^2 I$. Multiple features are stacked into a single update matrix.

4.4.4 Measurement Compression and Gating

Before applying the EKF update, a further QR compression reduces the stacked system to at most n rows, reducing the computational cost of the Kalman gain solve. Each feature's contribution is also validated by a Mahalanobis distance gate: the innovation is accepted only if

$$\tilde{r}^T S^{-1} \tilde{r} \leq \chi^2_{\text{mult}} \cdot \chi^2_{0.95, \text{dof}}$$

where $S = \tilde{H}_x P \tilde{H}_x^T + \tilde{R}$ is the innovation covariance and χ^2_{mult} is a configurable multiplier. This gate rejects features whose reprojection error is inconsistent with the current uncertainty, providing robustness to triangulation outliers and tracking errors.

4.4.5 EKF Update

The standard EKF update is applied to the accepted compressed system:

$$\begin{aligned} K &= P H_x^T (H_x P H_x^T + R)^{-1} \\ \delta x &= K \tilde{r} \\ P &\leftarrow (I - K H_x) P \end{aligned}$$

The state increment δx is applied on the appropriate manifold for each variable type. Orientations are updated via left-multiplicative quaternion composition; all vector quantities use additive

increments. Numerical stability is maintained by adding a small diagonal jitter to S before solving and by enforcing covariance symmetry after each update.

4.5 SLAM Landmark Management

4.5.1 *Delayed Initialisation*

Long-track features that meet quality criteria are promoted from marginalised MSCKF tracks to persistent SLAM landmarks in the state vector. A candidate feature is triangulated and refined as described in Section 4.3.4. If the triangulation passes the condition and depth checks, a Landmark variable is added to the state via the augmented initialisation procedure, which computes the linearised relationship between the new landmark block and the existing state variables and expands the covariance accordingly.

4.5.2 *Landmark Representations*

Six coordinate representations are supported for SLAM landmarks, selectable per-feature type:

- GLOBAL_3D: Euclidean coordinates in the global frame (3 DOF).
- GLOBAL_FULL_INVERSE_DEPTH: Full inverse-depth in global frame (6 DOF).
- ANCHORED_3D: Euclidean coordinates relative to an anchor clone.
- ANCHORED_FULL_INVERSE_DEPTH: Inverse-depth relative to anchor (6 DOF).
- ANCHORED_MSCKF_INVERSE_DEPTH: Compact anchored form (3 DOF) used for MSCKF-like SLAM.
- ANCHORED_INVERSE_DEPTH_SINGLE: Single-depth parameterisation (1 DOF) with cached bearing.

4.5.3 *Anchor Management*

When the oldest clone in the sliding window is marginalised, any SLAM landmark anchored to that clone must be re-anchored to a newer clone. This is achieved by transforming the landmark coordinates through the relative pose between old and new anchor frames, and propagating the

covariance change through the corresponding Jacobian. This operation preserves filter consistency under anchor transitions.

4.6 Zero-Velocity Update (ZUPT)

The Zero-Velocity Update provides strong observability constraints during static or near-static vehicle conditions. When the vehicle is stationary, the true velocity is zero and all IMU measurements reflect only bias and gravity. The ZUPT exploits this by formulating gyroscope and accelerometer residuals that enforce the stationary assumption in the EKF framework.

4.6.1 Residual Model

Over a selected IMU segment of duration Δt , the gyroscope residual enforces zero angular rate:

$$r_{\omega} = -(\sqrt{\Delta t} / \sigma_{\omega}) \hat{\omega}$$

The accelerometer residual enforces consistency with the gravity projection:

$$r_a = -(\sqrt{\Delta t} / \sigma_a) (\hat{a} - R_{\{GtoI\}} g)$$

Both residuals are normalised by the measurement noise density to form a dimensionless innovation. The Jacobians of these residuals with respect to the state (orientation, velocity, biases) are assembled into a compressed update system using the same QR procedure as the MSCKF updater.

4.6.2 Acceptance Gating

The ZUPT is applied only when three conditions are jointly satisfied: the chi-square innovation test passes, the estimated velocity is below a configured magnitude threshold, and the optical flow disparity across a reference image pair is below a maximum disparity threshold. The disparity condition prevents the ZUPT from firing during slow rotational motion where the IMU signal

might appear stationary but the vehicle is in fact moving. A force mode exists for startup scenarios where the disparity check is required to pass while the chi-square is relaxed.

4.7 Hover-Aware FIFO/LIFO Supervisory Estimation Framework

This section describes the primary original contribution of this thesis: a supervisory estimation framework that adapts the VIO pipeline's update strategy in response to detected low-excitation hover conditions. The motivation, detection criterion, mode switching logic, LIFO update semantics, and recovery behaviour are described in turn.

4.7.1 Motivation: Observability Degradation Under Hover

In standard MSCKF operation (FIFO mode), translational observability is derived from the parallax generated by camera motion between successive clone poses. When the UAV hovers with near-zero baseline, the parallax for all features collapses, and the nullspace-projected MSCKF constraint provides little useful information about translation or depth. Formally, the feature Jacobian H_f becomes nearly rank-deficient in the translational direction, and the projected constraint $\tilde{H}_x$ provides negligible information about the position components of the state. Applying a low-information update in this regime introduces numerical noise into the filter, potentially degrading the covariance structure without improving the estimate.

This motivates a mode switch: rather than continuing to push near-zero-information MSCKF constraints through the filter, the pipeline transitions to a Hover mode (LIFO) that halts clone augmentation, preserves the existing clone window context, and uses a different update strategy suited to the geometrically degenerate configuration.

4.7.2 Hover Detection Criterion

Hover is detected by computing a bearing residual statistic that quantifies the apparent displacement of visual features after compensating for the IMU-estimated rotation between consecutive frames. Given a set of normalised bearing vectors $\{b_{prev}\}$ and $\{b_{curr}\}$ observed in two consecutive frames, and the relative rotation $C_{\hat{q}}$ derived from IMU propagation:

$$r_i = \| b_{\{curr,i\}} - C_{\{\hat{q}\}} b_{\{prev,i\}} \|^2$$

This residual isolates the translational component of apparent feature motion: after removing the IMU-estimated rotation, remaining bearing displacement is attributable only to camera translation. Under pure hover, this residual should be near zero for all features.

4.7.3 *Baseline-Adaptive Inlier Filtering*

The bearing residuals are computed over an inlier set determined adaptively based on the estimated inter-frame baseline magnitude. When the baseline exceeds a configured threshold `hover_baseline_threshold`, an epipolar geometry filter is applied: correspondences inconsistent with the estimated essential matrix are removed before the residual is computed. This prevents large-baseline motion from producing spuriously small residuals due to degenerate feature configurations. When the baseline is below the threshold, a pure bearing inlier filter is used, retaining only features whose residual falls below `hover_bearing_inlier_threshold`.

The scalar hover decision statistic d_k is then the mean residual of the inlier set (or the median for robustness when the inlier count is small):

$$d_k = \text{mean}_{\{i \in \text{inliers}\}}(r_i)$$

The binary hover decision is:

$$\xi_k = 1 \text{ if } d_k < \tau_{\text{hover}}, \text{ } 0 \text{ otherwise}$$

where $\tau_{\text{hover}} = \text{hovering_threshold}$ is a configured scalar threshold.

4.7.4 Temporal Smoothing

To prevent mode flapping from transient feature tracking noise, the binary decisions ξ_k are accumulated in a fixed-length deque of size `hover_smoothing_required`. A mode transition from FIFO to LIFO is only triggered when all entries in the deque are 1, indicating consistent hover detection over multiple consecutive frames. If the deque has not yet accumulated enough history (such as at startup), the last stable decision is retained. This hysteresis mechanism significantly reduces spurious mode transitions at the boundary of the hover threshold.

4.7.5 FIFO→LIFO Entry Guards

Even when the smoothed hover decision is positive, the transition from FIFO to LIFO is guarded by two additional safety conditions:

- Velocity guard: The current estimated velocity magnitude must not exceed `hover_entry_max_velocity`. This prevents LIFO entry during slow translational motion that might produce low bearing residuals.
- Baseline guard: The estimated inter-frame camera baseline must not exceed `hover_entry_max_baseline`. This ensures the system is genuinely in a near-static configuration rather than a brief pause between motions.

These guards form the complete FIFO→LIFO transition condition: hover detected AND velocity below threshold AND baseline below threshold.

4.7.6 LIFO→FIFO Exit Conditions

The system exits LIFO and returns to FIFO mode under any of three conditions, checked in priority order:

1. The smoothed hover decision drops to 0, indicating resumed translational motion.
2. The estimated velocity magnitude exceeds `hover_exit_velocity_threshold`, providing an emergency override independent of the visual detector.
3. The total time in LIFO mode exceeds `hover_max_duration_sec`, a safety time limit preventing indefinite LIFO operation.

The state machine implementing this logic is shown in Figure 4.2.

Figure 4.2: FIFO/LIFO supervisory state machine. Transitions are triggered by the smoothed hover decision ξ and the velocity/baseline guard conditions. The max-duration timer provides an unconditional fallback to FIFO.

4.7.7 LIFO Update Semantics: Backward-Chain Feature Association

Within LIFO mode, the absence of new clone augmentation means that no new pose variable is added to the state on each camera frame. Feature observations are instead associated against the frozen clone window using a backward-chain matching strategy designed to tolerate the temporal repetition of near-identical observations under hover.

For each active feature, the system maintains a quality score $q \in [0, 1]$ that evolves as:

$$\begin{aligned} q &\leftarrow q + \Delta q_{\text{match}} \quad (\text{on successful geometric match}) \\ q &\leftarrow q - \Delta q_{\text{miss}} \quad (\text{on failed match}) \\ q &\leftarrow q + \Delta q_{\text{ORB}} \quad (\text{on successful ORB descriptor match, optional fallback}) \end{aligned}$$

Features whose quality score falls below $q_{\text{min_keep}}$ are pruned from the active LIFO chain. Only features above $q_{\text{min_output}}$ are passed to the MSCKF updater within LIFO mode. This quality-gated chain prevents the accumulation of noise from intermittently tracked features under blur or illumination change during hover.

Association is formalised as an assignment problem over the feature set, solved using the Hungarian algorithm with a composite cost combining normalised reprojection error and descriptor distance:

$$\text{cost}(i,j) = w_{\text{reproj}} \cdot \rho_{\{\text{reproj}\}}(i,j) + w_{\text{desc}} \cdot \rho_{\{\text{desc}\}}(i,j)$$

where $\rho_{\{\text{reproj}\}}$ is the normalised reprojection cost and $\rho_{\{\text{desc}\}}$ is the normalised ORB Hamming distance. An optional mutual consistency check rejects assignments where the nearest-neighbour ratio exceeds a threshold, further suppressing false associations.

4.7.8 Deferred Covariance Strategy

A key design choice in LIFO mode is the decoupling of mean and covariance updates. In each LIFO step, the MSCKF update is applied to the state mean (position, orientation, velocity, biases) as normal. However, the covariance is not updated forward from this step; instead, the pre-update covariance snapshot is restored after computing the mean increment. The measurement data is accumulated in a deferred measurement buffer.

The rationale for this strategy is that, under near-zero-baseline hover, the MSCKF Jacobians $\tilde{H}_x$ are geometrically degenerate in the translational directions. Applying repeated near-zero-information updates to P would introduce numerical noise into the covariance without genuine information gain, potentially rendering P overconfident and causing filter inconsistency on the return to FIFO mode. By preserving the covariance structure and applying accumulated measurements only on the FIFO re-entry step (Section 4.7.9), the filter maintains a conservative and consistent uncertainty estimate across the hover period.

It is acknowledged that strict simultaneous mean-covariance coupling is relaxed by this strategy. This is an engineering approximation validated empirically, and its impact on filter consistency should be evaluated using Normalised Estimation Error Squared (NEES) analysis in the experimental section.

4.7.9 FIFO Re-Entry and Covariance Reconciliation

When the mode machine triggers a LIFO→FIFO transition, the deferred measurement buffer accumulated during the hover period is used in a covariance reconciliation step. This step applies the stacked hover observations to the restored covariance, effectively condensing the information gathered during hover into a single batch update before resuming the standard FIFO clone augmentation pipeline. Clone augmentation then resumes normally, and the subsequent FIFO

update benefits from both the corrected mean (maintained throughout LIFO) and the reconciled covariance.

4.7.10 Recovery Coupling: Update Failures and Forced Hover

The pipeline additionally maintains a failure counter that increments each time the MSCKF update is rejected (by chi-square gate) or produces fewer than the minimum required rows. When this counter exceeds a configured threshold, two recovery behaviours are activated:

- **Hover override:** A forced entry into LIFO mode is permitted even if the smoothed hover decision is not fully positive. This handles situations where degenerate geometry is causing update failures that do not produce a clean hover signal.
- **Forced ZUPT:** A zero-velocity update is attempted if stationarity indicators (disparity, velocity) suggest the vehicle is truly stationary, even if the standard ZUPT gating would not have triggered.

This recovery coupling links the update quality monitor to the mode controller, providing a fallback path that prevents filter divergence during sustained tracking difficulties.

Table 4.1: Summary of FIFO vs. LIFO Mode Characteristics

Property	FIFO Mode	LIFO Mode
Clone augmentation	Yes — new clone per frame	No — clone window frozen
Propagation type	propagate_and_clone	propagate_only
MSCKF update	Full standard update	State-only, deferred covariance
Feature association	Forward temporal matching	Backward chain, quality-scored
Covariance update	Applied each step	Snapshot restored; batch on exit
SLAM updates	Yes	Suspended
Entry condition	Default / LIFO exit	Hover detected + guards pass
Exit condition	—	Motion resumed / timer / velocity

4.8 Inertial Initialisation

The filter cannot begin estimation until the initial orientation, velocity, and IMU biases are determined. A static initialisation procedure is implemented that monitors both IMU statistics and visual feature disparity to gate the initialisation attempt.

4.8.1 Static Initialisation Gating

The initialiser accumulates a window of IMU samples of length `init_window_time` seconds. The window is split into two halves, and the accelerometer variance is computed in each half. Initialisation is attempted only when the variance in both halves is below `init_imu_thresh`, confirming that the vehicle is stationary, AND the visual disparity computed from the feature database is below `init_max_disparity`, confirming the camera is also stationary. This dual gating prevents false initialisation during slow oscillatory motion.

4.8.2 Gravity Alignment and Bias Estimation

Under static conditions, the mean accelerometer reading $\bar{b}_a$ equals the gravity vector projected into the IMU frame: $\bar{b}_a \approx R_{\{GtoI\}} g$. The gravity-aligned orientation is recovered using a Gram-Schmidt orthogonalisation procedure that constructs a rotation matrix aligning the measured acceleration direction with the known gravity vector. The initial biases are then:

$$\begin{aligned} b_g &= \bar{b}_\omega \quad (\text{mean gyroscope reading under stationary condition}) \\ b_a &= \bar{b}_a - R_{\{GtoI\}} g \end{aligned}$$

The initial state covariance is set using fixed configured uncertainty values for each variable block, reflecting the uncertainty in the gravity-alignment procedure and the variance of the bias estimates.

4.9 Configuration and Calibration Loading

All pipeline parameters are loaded from three YAML files in a designated configuration folder: `estimator_config.yaml` (algorithmic parameters), `kalibr_imu_chain.yaml` (IMU calibration), and `kalibr_imucam_chain.yaml` (camera calibration and IMU-camera extrinsics). The loader parses each file, constructs the corresponding options objects, and calls a final synchronisation step that copies loaded calibration values into the state initialisation structures. This design allows the pipeline to support different sensor platforms by swapping only the configuration folder without code changes.

The FIFO/LIFO hover framework introduces its own configuration namespace with parameters governing detection sensitivity (`hovering_threshold`, `hover_smoothing_required`), entry/exit guards (`hover_entry_max_velocity`, `hover_entry_max_baseline`, `hover_exit_velocity_threshold`), duration limits (`hover_max_duration_sec`), and the full LIFO chain quality evolution parameters (`lifo_quality_init`, `lifo_quality_inc_match`, `lifo_quality_dec_miss`, `lifo_max_misses`, and associated matching thresholds).

CHAPTER 5: IMPLEMENTATION NOTES (OUTLINE)

Chapter 5 should document the Python-specific implementation decisions that are distinct from the mathematical methodology. The recommended content is:

- Python port rationale: why Python was chosen and what constraints this imposed (GIL, NumPy/SciPy vectorisation, ROS Python bindings).
- JPL quaternion library (type/quaternion_ops.py): implementation of Lie algebra utilities `exp_so3`, `log_so3`, `Jr_so3` as direct NumPy operations.
- Covariance block management: Python dictionary-based state ordering vs. compile-time C++ type registration in OpenVINS; how `StateHelper.EKFPropagation` handles variable-length block structures using dynamic index tracking.
- Threading model: Python `threading.Lock` usage for camera queue and ZUPT buffer; implications of the GIL on IMU callback throughput.
- Camera model implementations: `CamRadtan` (radial-tangential distortion) and `CamEqui` (equidistant/fisheye) as separate classes inheriting from `CamBase`, and how projection/unprojection is used in Jacobian chains.
- Dynamic initialiser status: `dynamic.py` is present but empty; only static initialisation is available in the current implementation.
- FEJ (First-Estimates Jacobians): optional consistency improvement enabled via `state_options.do_fej` flag, using stored first-estimate values in `JPLQuat` and `Landmark` objects.

CHAPTER 6: EXPERIMENTS AND EVALUATION (OUTLINE)

Chapter 6 must demonstrate the system's performance quantitatively. The following structure is recommended:

6.1 Evaluation Datasets

Standard VIO benchmark datasets should be used. The EuRoC MAV dataset is the primary reference for this type of pipeline. For hover-specific evaluation, sequences with sustained low-motion periods are most informative.

6.2 Evaluation Metrics

- Absolute Trajectory Error (ATE): RMSE between estimated and ground-truth trajectory after optimal rigid alignment. Primary accuracy metric.
- Relative Trajectory Error (RTE): Drift measured over fixed-length trajectory segments. Measures local consistency.
- NEES (Normalised Estimation Error Squared): Evaluates filter consistency. Ideally $\text{NEES} \approx n$ (state dimension) for a consistent filter.
- Hover mode occupancy: Percentage of frames processed in LIFO mode. Useful for system characterisation.
- Update acceptance rate: Fraction of MSCKF updates accepted by the chi-square gate.

6.3 Ablation Study

The following ablation conditions are recommended to isolate the contribution of the FIFO/LIFO framework:

- Baseline: Full pipeline with FIFO/LIFO and ZUPT enabled.
- No-LIFO: FIFO only, LIFO disabled. Shows the benefit of hover mode.
- No-ZUPT: LIFO enabled, ZUPT disabled. Separates ZUPT contribution from hover handling.
- No-LIFO and No-ZUPT: Pure MSCKF baseline, closest to canonical OpenVINS.

6.4 Comparison

Where possible, comparison against the original OpenVINS C++ implementation on the same sequences provides a reference for the cost of the Python reimplementations. The goal is not to outperform OpenVINS in baseline accuracy but to demonstrate that the hover extension reduces drift on hover-heavy sequences compared to the pipeline without it.

REFERENCES (PLACEHOLDER)

The following references are recommended as minimum required citations for this thesis:

4. Mourikis, A. I. and Roumeliotis, S. I. (2007). A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation. ICRA.
5. Geneva, P. et al. (2020). OpenVINS: A Research Platform for Visual-Inertial Estimation. ICRA.
6. Forster, C. et al. (2017). On-Manifold Preintegration for Real-Time Visual-Inertial Odometry. IEEE TRO.
7. Huang, G. et al. (2010). Observability-based Rules for Designing Consistent EKF SLAM Estimators. IJRR.
8. Lucas, B. D. and Kanade, T. (1981). An Iterative Image Registration Technique. IJCAI.
9. Kuhn, H. W. (1955). The Hungarian Method for the Assignment Problem. Naval Research Logistics.
10. Li, M. and Mourikis, A. I. (2013). High-Precision, Consistent EKF-Based Visual-Inertial Odometry. IJRR.
11. Furgale, P. et al. (2013). Unified Temporal and Spatial Calibration for Multi-Sensor Systems. IROS.
12. Hartley, R. and Zisserman, A. (2004). Multiple View Geometry in Computer Vision. Cambridge University Press.

APPENDIX A: KEY MATHEMATICAL DERIVATIONS

A.1 JPL Quaternion Left-Multiplicative Update

For a small rotation error $\delta\theta \in \mathbb{R}^3$, the perturbed quaternion is:

$$\begin{aligned}\delta q &= [\tfrac{1}{2} \delta\theta_x, \tfrac{1}{2} \delta\theta_y, \tfrac{1}{2} \delta\theta_z, 1]^T \text{ (unnormalised)} \\ q^{+} &= \text{quatnorm}(\delta q \otimes q)\end{aligned}$$

This convention ensures that the error-state for orientation is a 3-vector $\delta\theta$ rather than a 4-vector, maintaining the correct degrees of freedom in the covariance matrix.

A.2 Hover Detector Baseline Estimation

The baseline between two camera frames is estimated from the clone poses already in the state.

Given the most recent clone pose ($R_{\{GtoI\}}^{\{prev\}}, p_{I}^{\{prev\}}$) and the current propagated IMU pose ($R_{\{GtoI\}}^{\{curr\}}, p_{I}^{\{curr\}}$), the camera baseline magnitude in the global frame is:

$$b_{cam} = \| p_{I}^{\{curr\}} - p_{I}^{\{prev\}} \| + \| R_{\{ItoC\}}(p_{I}^{\{curr\}} - p_{I}^{\{prev\}}) \| / 2$$

This estimate is used both for the adaptive inlier filter selection and for the FIFO→LIFO entry baseline guard.